Database for Data Science



Databases can be broadly categorized into three types: relational, non-relational databases and graph database

SQL (Relational) Databases:

- Relational databases, such as MySQL and PostgreSQL, use a structured query language (SQL) to manage data stored in tables with rows and columns.
- These databases are well-suited for data that can be easily organized into a tabular format and are useful for data warehousing, business intelligence, and other reporting applications.

NoSQL (Non-relational) Databases:

- NoSQL databases, such as MongoDB and Cassandra, are designed to handle large amounts of unstructured data and support different data models, such as document, key-value, and graph.
- These databases are useful for big data and real-time applications, such as real-time analytics, social media, and gaming.

- Graph databases, such as Neo4j and Amazon Neptune, are used to store and query data in a graph format, where nodes and edges represent entities and relationships.
- These databases are useful for applications that involve complex relationships, such as fraud detection, recommendation systems, and social network analysis.

Database Business intelligence process

ETL and ELT

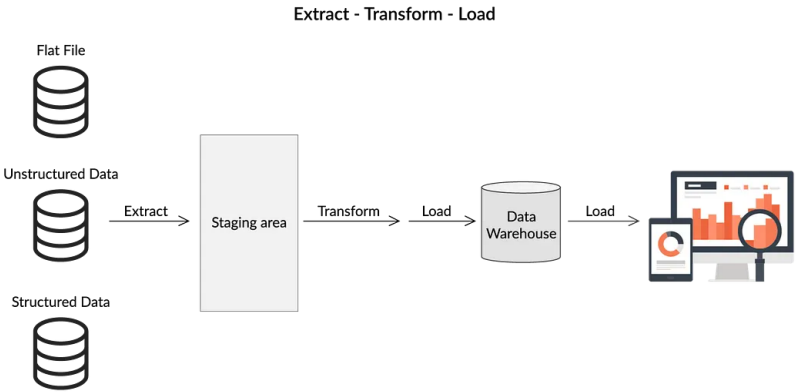
The process of importing and obtaining data for immediate use and storage in a database is known as data ingestion.

There are two common methods for creating a data ingestion pipeline: ETL and ELT.

ETL (Extract, Transform, Load):

In this method, data is first extracted from various sources, then transformed into a suitable format, and finally loaded into the database.

This approach ensures that the data is cleaned, formatted, and ready for analysis before it is stored.



ELT (Extract, Load, Transform):

This method reverses the last two steps.

Data is first extracted and loaded directly into the database, and the transformation process occurs afterwards within the database itself.

This approach can be more efficient for handling large volumes of data and allows for more flexibility in data processing.

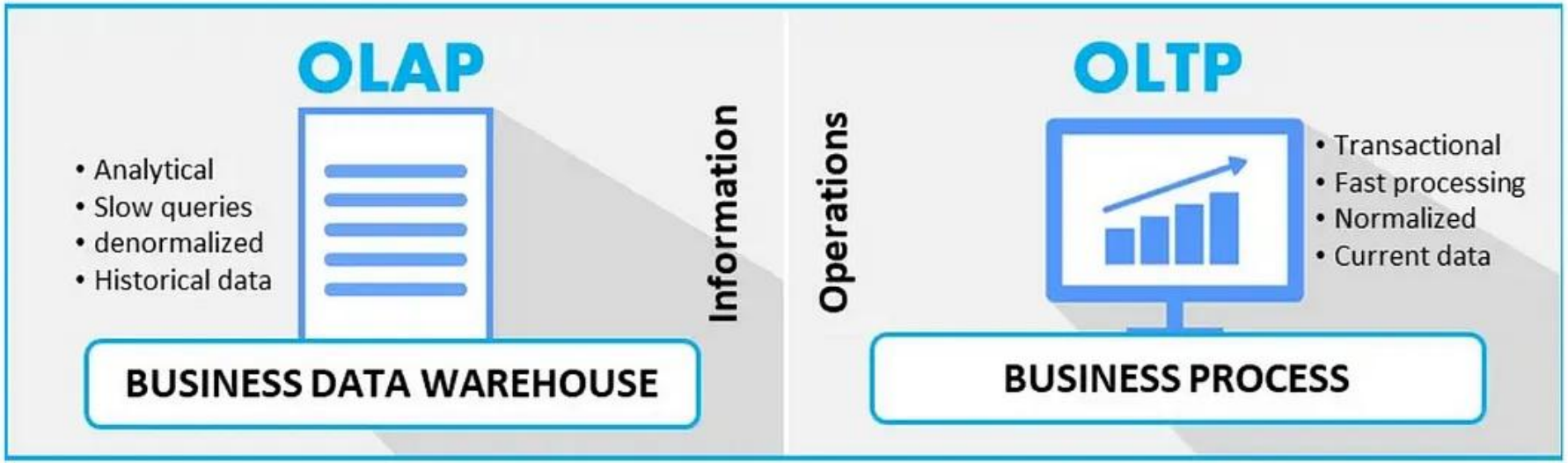
Extract: Connecting to a specific data source and extracting the data.

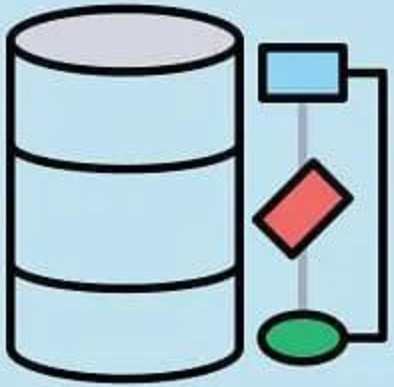
Transform: Modifying the extracted data to standardize and format it appropriately.

Load: Pushing the data into storage.

- **ETL (Extract, Transform, Load)**: Data filtering and transformation occur before the data is stored.
- **ELT (Extract, Load, Transform)**: Data is stored first, and filtering and transformation occur afterwards.

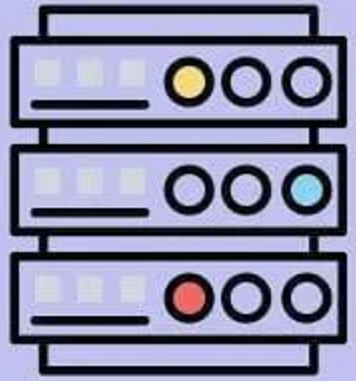
OLAP Vs OLTP


OLAP
• Analytical
• Slow queries
• denormalized
• Historical data
BUSINESS DATA WAREHOUSE
Information
Operations
OLTP
• Transactional
• Fast processing
• Normalized
• Current data
BUSINESS PROCESS



DBMS

VS



RDBMS

What is a Relational Database Management System (RDBMS)?

An RDBMS is a database management system that stores data in a structured format, using tables (or relations) composed of rows and columns.

Each table holds data about a specific entity, and relationships between these tables allow data to be linked together, creating a powerful and flexible data model.

Features of RDBMS

An RDBMS addresses the drawbacks of file-based storage systems by organizing data into related tables, following a relational model.

- **Primary Keys**: Uniquely identify each row in a table.
- **Composite Primary Keys**: Combine two or more keys to uniquely identify each row when a single key is insufficient.
- **Foreign Keys**: Fields in one table that act as primary keys in another, establishing relationships between tables.

Why RDBMS

1. Data Integrity and Consistency
2. Structured Data Handling
3. Scalability and Performance
4. Data Security and Access Control
5. Seamless Integration with Analytical Tools

Normalization in RDBMS

Normalization reduces data redundancy and ensures data integrity by structuring a database in a way that minimizes duplication.

1. **First Normal Form (1NF)**:
    - Each cell must contain a single value.
    - Each row must be uniquely identifiable.

2. Second Normal Form (2NF):

- Must be in 1NF.
- No attribute should be partially dependent on the primary key.

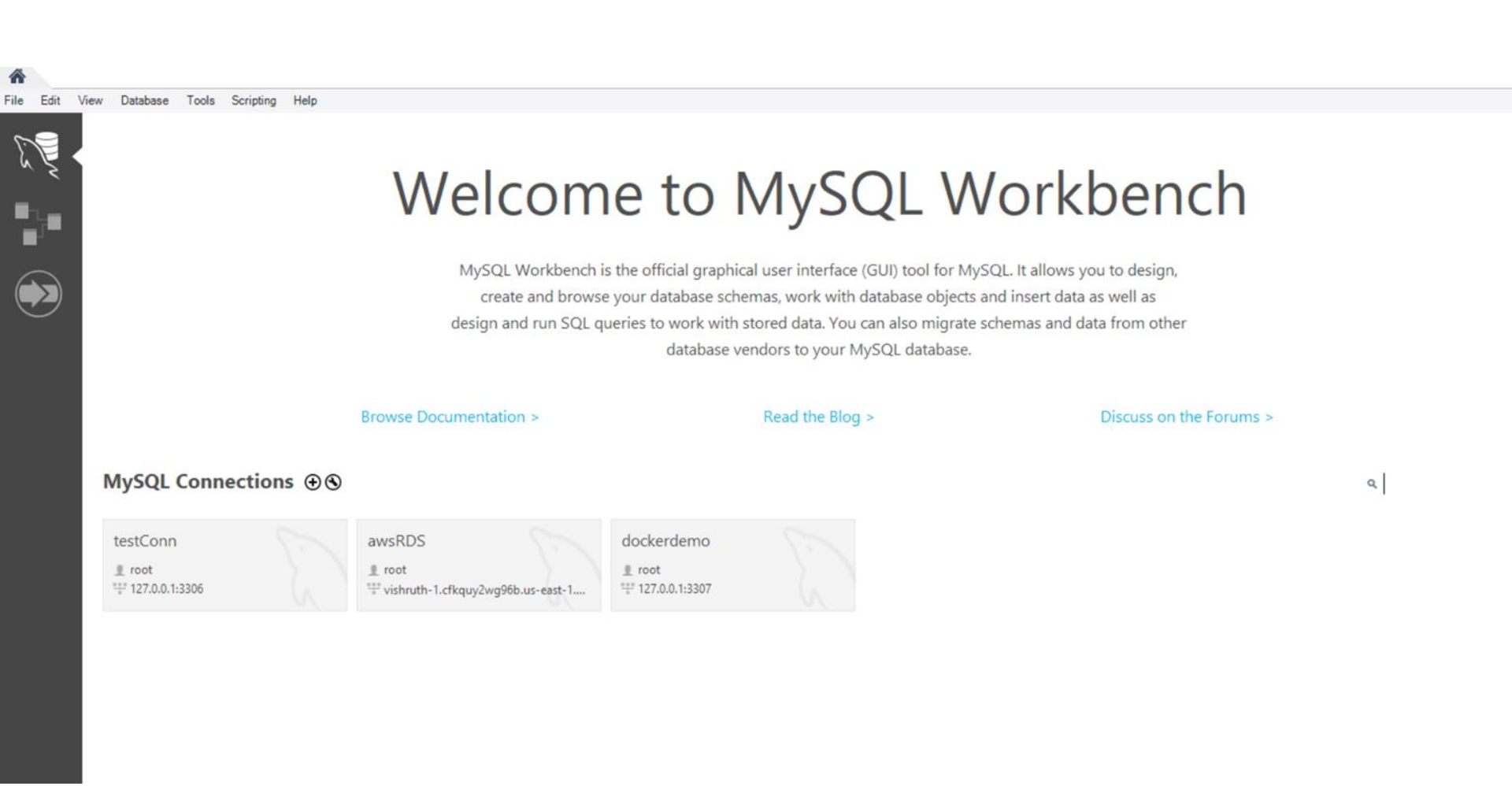
3. Third Normal Form (3NF):

- Must be in 2NF.
- No transitive dependency should exist within a table.

Use Cases of RDBMS in Data Science

1. Customer Analytics
2. Fraud Detection
3. Operational Efficiency
4. Healthcare Data Management


MySQL™


File Edit View Database Tools Scripting Help
Welcome to MySQL Workbench
MySQL Workbench is the official graphical user interface (GUI) tool for MySQL. It allows you to design, create and browse your database schemas, work with database objects and insert data as well as design and run SQL queries to work with stored data. You can also migrate schemas and data from other database vendors to your MySQL database.
Browse Documentation >
Read the Blog >
Discuss on the Forums >
MySQL Connections
testConn
root
127.0.0.1:3306
awsRDS
root
vishruth-1.cfkquy2wg96b.us-east-1....
dockerdemo
root
127.0.0.1:3307

MySQL Join

•A **JOIN** is a method to combine data from two or more tables in a **database** based on a related column between them.

•JOINs allow us to retrieve related data from multiple tables in a single query, avoiding the need for numerous separate queries.

•JOINs help maintain **referential integrity** by ensuring that relationships between tables are respected and **data consistency** is preserved.

•By allowing the use of normalized database structures (where data is split into different tables to reduce redundancy), JOINs help in efficiently **managing** and **organizing** data.

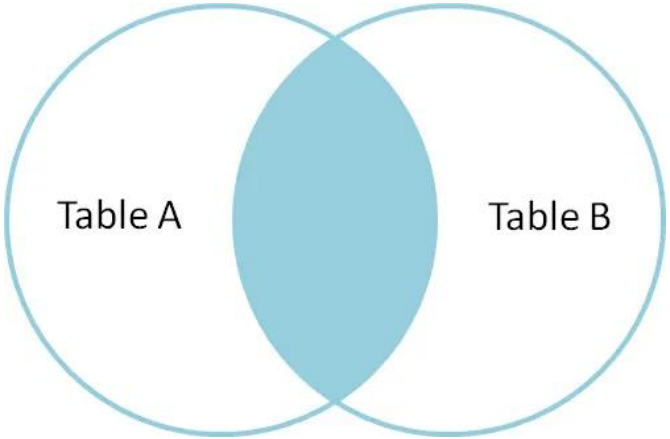
•JOINs are often used in conjunction with GROUP BY and aggregation functions (like **COUNT, SUM, AVG**) to perform comprehensive data analysis.

Syntax:

SELECT column_names FROM table1 INNER JOIN table2
ON table1.common_column = table2.common_column;

Inner Join

It Returns records that have matching values in both tables.



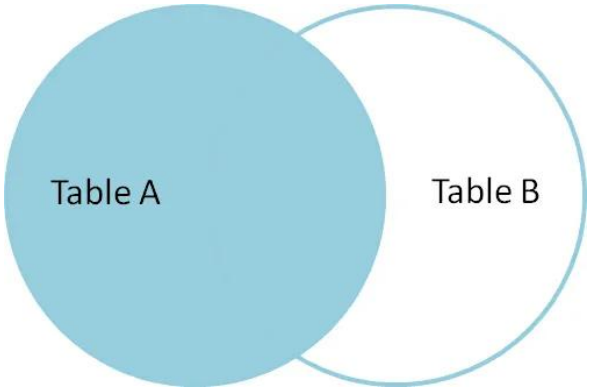
Syntax:

```
SELECT table1.column1,table1.column2,table2.column1,.... FROM table1  INNER
JOIN
 table2 ON  table1.matching_column = table2.matching_column;
```

2. LEFT JOIN

It returns all records from the Left table and matched records from the Right table.

If there is no match, then NULL values are returned for Right table columns.



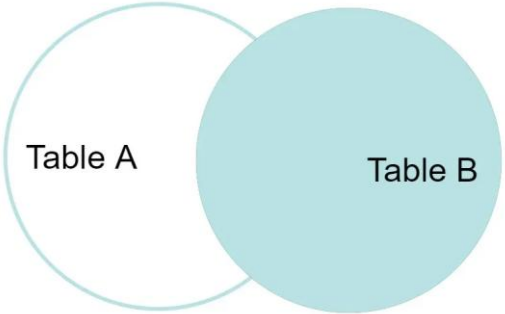
Syntax:

```
SELECT table1.column1,table1.column2,table2.column1,....
FROM table1
LEFT JOIN table2
ON table1.matching_column = table2.matching_column;
```

3. RIGHT JOIN

It Returns all the rows from the right table and the matched rows from the left table.

NULL values will be returned for columns from the left table when there are no matches.

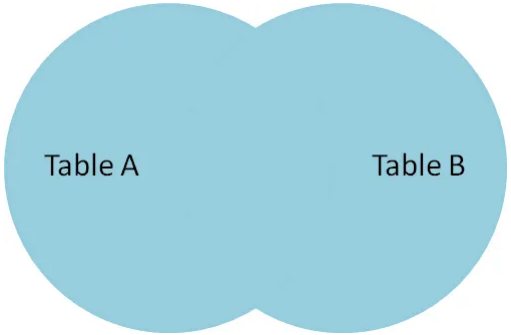


Syntax

```
SELECT table1.column1,table1.column2,table2.column1,....
FROM table1
RIGHT JOIN table2
ON table1.matching_column = table2.matching_column;
```

4. FULL JOIN

It Returns all records when there is a match in either the left or the right table.
In case of no match, NULL values are returned for columns that have no match in either table.



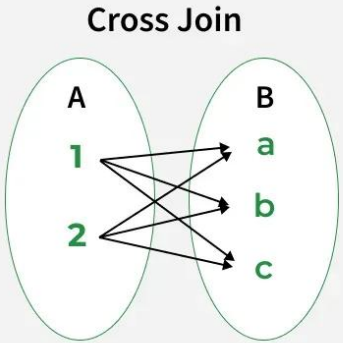
Syntax

```
SELECT table1.column1,table1.column2,table2.column1,....
FROM table1
FULL JOIN table2
ON table1.matching_column = table2.matching_column;
```

5. CROSS JOIN

It Returns the Cartesian product of two tables.

Matches every row of one table with every row of another table.



Syntax:

```
SELECT * FROM table1 CROSS JOIN table2;
```

6. SELF JOIN

A **self join** is a type of join in which a table is joined to itself.

This is useful when you need to compare rows within the same table, such as relating employees to their managers in an organizational hierarchy.

How to Join Three or More Tables in SQL?

1. Using SQL Joins

2. **Using parent-child relationship:**

SQL Subquery

A subquery in SQL is a query nested inside another SQL query.
It allows complex filtering, aggregation and data manipulation by using the result of one query inside another.

StudentID	Name	Score
1	Alice	88
2	Daniel	92
3	Maria	79

```
>SELECT * FROM Students WHERE Score >  ( SELECT AVG(Score) FROM Students );
```

StudentID	Name	Score
1	Alice	88
2	Daniel	92

SQL Clauses for Subqueries

Clauses that can be used with subqueries are:

•Filters rows in the outer query based on the results returned by the WHERE clause.

•Treats the subquery as a temporary (derived) table that can be queried like a normal table using the FROM clause.

•Filters grouped or aggregated results using values produced by the HAVING clause.

Types of Subqueries

1. Single-Row Subquery

A single-row subquery is a subquery that returns only one value.

- Returns exactly one row as the result.
- Commonly used with comparison operators such as =, >, <

2. Multi-Row Subquery

A multi-row subquery is a subquery that returns more than one value.

- Returns multiple rows as the result.
- Requires operators that can handle multiple values, such as IN, ANY or ALL

3. Correlated Subquery

A correlated subquery is a subquery that depends on the outer query for its values.

•A dependent subquery: it references columns from the outer query.

•Executed once for each row of the outer query, making it slower for large datasets.

SQL Indexes

Indexes are specialized data structures that improve database query performance by creating an optimized pathway to locate data.

- It speed up queries (SELECT, JOIN, WHERE, ORDER BY).
- Reduce disk I/O and improve efficiency in large tables.
- Ensure data integrity with unique indexes.

Note : primary keys are automatically indexed

1. Creating an Index

1.1. Single Column Indexes

A single-column index is created on just one column.

It’s the most basic type of index and helps speed up queries when you frequently search, filter or sort by that column.

Syntax:

```
CREATE INDEX index_name ON TABLE column;
```

1.2. Multi Column Indexes

A multi-column index is created on two or more columns. It improves performance when queries filter or join based on multiple columns together.

Syntax:

```
CREATE INDEX index ON TABLE (column1, column2,.....);
```

Unique Indexes

A unique index ensures that all values in a column (or combination of columns) are unique preventing duplicates and maintaining data integrity.

Syntax:

```
CREATE UNIQUE INDEX index_name ON table_name (column_name);
```

Confirming and Viewing Indexes

We can view all the indexes in a database to understand which ones are in use and confirm their structure. In SQL, the following query helps us see the indexes for a given table:

Syntax:

```
SHOW INDEXES FROM table_name;
```

3. Removing an Index

Indexes take up storage and add overhead on write operations (INSERT, UPDATE, DELETE). If an index is no longer needed, it can be removed.

Syntax:

```
DROP INDEX index_name;
```

Altering an Index

If an index requires adjustments, such as reorganizing or rebuilding, it can be altered without affecting the data.

This is useful for optimizing index performance as tables grow larger.

Syntax:

```
ALTER INDEX IndexName ON TableName REBUILD;
```


Database Best Practices

1. Designing a Scalable Data Architecture

- A **scalable data architecture** ensures that data pipelines can handle increasing **data volume, velocity, and variety** without performance degradation.
- **Choosing the Right Storage Strategy -** The type of storage you choose depends on the structure of your data:

Data Modelling Best Practices

Data modeling impacts **query performance, data retrieval, and storage efficiency**.

Here's how to do it right:

- **For Transactional Databases:**

•**Normalize the schema** (1NF, 2NF, 3NF) to avoid redundant data and ensure data integrity.

•Use **indexes** to speed up searches but avoid excessive indexing, which can slow down writes.

- **For Analytical Databases:**

•Use **denormalized structures** (Kimball's **Star Schema** or Snowflake Schema) to optimize query performance.

•Store aggregated data in **materialized views** to speed up queries.

1. Schema design best practices:

Model from use-cases, not just entities: Design tables around how data will be queried (OLTP vs OLAP, batch vs realtime, key dimensions/facts).

2. Use proper primary keys

3.Choose correct data types: Use appropriate numeric, date/time, and string types; avoid over-wide VARCHAR, store booleans properly, avoid “everything as text”.

4. Enforce integrity in the DB: Use NOT NULL, CHECK, UNIQUE, and FOREIGN KEY constraints so pipelines can trust the data model.

5. Plan for evolution: Version schemas, use migrations, and avoid breaking changes; document changes for downstream consumers.

SQL query best practices:

1. Correctness and readability first: Write clear SQL (aliases, formatting, meaningful names) then optimize queries that are slow or frequent.
2. Avoid **SELECT * in pipelines**: Project only needed columns to cut I/O, network, and improve optimizer choices.
3. Filter early and precisely: Use selective WHERE clauses, avoid returning unneeded rows; paginate or LIMIT for UIs and exploratory queries.
4. Use joins intentionally: Prefer explicit JOIN ... ON over old-style comma joins; pick correct join type (INNER vs LEFT) to match business meaning.

3. Index design best practices:

- Always index foreign keys and main search keys: Common join keys (student_id, course_id) and high-cardinality filter columns (email, timestamp, id) are prime candidates.
- Follow the left-most prefix rule; place equality filters first, then ranges, then sort/group columns; start with the most selective columns where possible.

4. Document data contracts:
Clearly document schemas, meanings, and SLAs so downstream analysts know how to use the data safely.